

Axion Strings AMR

Getting Started on a Cluster

September 23, 2026

Who this is for. You can already `ssh` into a cluster and `scp/rsync` files back and forth, but you don't need to be a systems person to follow this. Every command you need is given in full — copy, paste, adjust the few things flagged as “change this”.

Contents

1	What you're running	2
2	Step 1: Get the code onto the cluster	2
3	Step 2: Build it on the cluster	2
3.1	Load a compiler and MPI	2
3.2	Tell the build where FFTW is	3
3.3	Build	3
3.4	Ranks and threads: how to split up your CPUs	3
4	Step 3: Organise your files on the cluster	4
5	Step 4: Set up a run — the parameter file	4
5.1	The parameters you'll actually touch	4
5.2	The <code>fourier_relaxed</code> companion parameters	5
5.3	The handoff from relaxation to the main run	6
5.4	A note on <code>amr.max_grid_size</code>	7
5.5	Scaling up to a bigger test	7
5.6	A worked example: a moderately big cluster run	7
6	Less commonly changed options	8
6.1	How often refinement is re-checked, and its safety buffer	8
6.2	How sensitive the refinement trigger is (off by default)	8
6.3	Domain decomposition, the other half	9
6.4	Timestep safety margin	9
6.5	Cosmological expansion rate	9
6.6	Extra diagnostics	10
6.7	Checkpointing and restart	10
7	Step 5: Write and submit the SLURM script	10
8	Step 6: Watch it run	11
9	Step 7: Bring the results home	12
10	Step 8: Analyse it locally	12
11	Quick reference	13

1 What you're running

This is a 3D simulation code for axion cosmic strings, built on the AMReX/GRTeclyn adaptive-mesh-refinement (AMR) framework. “Adaptive mesh” means the simulation automatically uses a fine grid only where the physics needs it (near strings) and a coarse grid everywhere else, which is what makes bigger, more realistic runs affordable at all.

Practically, this means two things:

- One compiled program (an `.ex` file) runs every kind of simulation this project does — different physics comes from a plain-text *parameter file*, not different code.
- A run writes two very different kinds of output: a handful of small text files with the physics summaries you actually want (string density, energy, spectra, ...), and — only if you ask for them — large binary “plotfiles” with the full 3D field on disk. You will almost never want to move the plotfiles off the cluster; see Section 9.

2 Step 1: Get the code onto the cluster

The repository uses two git *submodules* (GRTeclyn and its dependency, AMReX) that must be checked out alongside it. If the cluster's login node has internet access, the cleanest approach is to clone directly there:

```
ssh you@cluster.example.edu
git clone --recurse-submodules https://github.com/<your-username>/
  axion_strings_adaptive.git
cd axion_strings_adaptive
```

If the cluster has no internet access from compute/login nodes (common on HPC systems), clone and prepare the submodules on your own machine first, then copy the whole tree over with `rsync` (much faster than `scp` for a big directory tree, and it can resume if interrupted):

```
# on your own machine
git clone --recurse-submodules https://github.com/<your-username>/
  axion_strings_adaptive.git
rsync -avz --progress axion_strings_adaptive/ you@cluster.example.edu:~/
  axion_strings_adaptive/
```

Do *not* copy a compiled executable from your laptop to the cluster — it was built for a different processor and operating system and will not run. Always build fresh on the cluster (Step 2).

3 Step 2: Build it on the cluster

3.1 Load a compiler and MPI

Clusters make software available through `module` commands rather than having everything installed system-wide. The exact module names differ between clusters, so start by looking:

```
module avail mpi
module avail fftw
```

On DESY's Maxwell cluster (the system this guide's SLURM template was written against) this looks like:

```
module load mpi/openmpi-x86_64
module load fftw
```

3.2 Tell the build where FFTW is

The code uses FFTW (for generating initial conditions), and the build needs to know where it's installed. After loading the `fftw` module, find its location:

```
module show fftw
```

Look for a line setting something like `FFTW_ROOT`, `EBROOTFFTW`, or a `-I.../include` flag — that directory (the one *containing* `include/` and `lib/`) is what you pass as `FFTW_HOME` below.

3.3 Build

This guide builds with multi-threading (OpenMP) switched on — each MPI rank uses several CPUs at once, not just one — since that's the setup you actually want for a real allocation:

```
cd axion_strings_adaptive/AxionStrings
make COMP=gnu FFTW_HOME=/path/from/module/show USE_OMP=TRUE -j8
```

- `COMP=gnu` tells it to use GCC, the standard choice on Linux clusters (on the project's own Mac, `COMP=llvm` is used instead — don't copy that part of any Mac instructions you see).
- `USE_OMP=TRUE` is what turns on multi-threading — see Section 3.4 for what this means for how you fill in the SLURM script. The bulk of the per-step cost (the physics update itself, and every energy/string-count diagnostic) is already written in a form AMReX itself splits across threads automatically when built this way — no further code changes needed to benefit from it.
- `-j8` builds with 8 parallel compile jobs; raise it if you have more cores available on the login node, but check the cluster's policy on running CPU-heavy work on login nodes first — if it's restricted, submit the build itself as a short batch job (Section 7, but with the `make` command in place of `mpirun`, and no MPI needed for the build). This `-j8` is about compiling the code faster and is unrelated to `USE_OMP` (which is about the compiled program's own later multi-threading).

A successful build takes a few minutes and ends with an executable named something like `AxionStrings3d.gnu.FLOAT.MPI.OMP.ex` sitting in `AxionStrings/` — note the `.OMP` tag, confirming multi-threading is compiled in. If it fails, the very last handful of lines of the error is almost always a missing module (wrong `FFTW_HOME`, or an MPI compiler wrapper not on `PATH`) — re-check Steps 2.1–2.2 before anything else.

3.4 Ranks and threads: how to split up your CPUs

You now have two independent dials for spreading a run across CPUs: `--ntasks` (MPI ranks, each a separate process with its own chunk of the domain) and `--cpus-per-task` (OpenMP threads, each rank splitting its own work further across several CPUs at once). Their product is the total number of CPUs the job reserves. A reasonable starting point is a handful of threads per rank (`--cpus-per-task=4` is used in the template below) rather than all-ranks-one-thread-each or one-rank-all-threads — adjust to taste once you have a feel for how your specific runs scale.

Watch out

A `USE_OMP=TRUE` build does nothing by itself until you also tell it how many threads to use, via the standard `OMP_NUM_THREADS` environment variable — **always set it to match `$SLURM_CPUS_PER_TASK`** in your submission script (the template in Section 7 already does this — keep that line if you copy it). Leaving it unset is not merely a missed optimisation: OpenMP then defaults to using *every* core it can see on the node, which badly oversubscribes the machine as soon as more than one MPI rank shares that node (the normal case). `--cpus-per-task` and `OMP_NUM_THREADS` always travel together — never set one without the other.

4 Step 3: Organise your files on the cluster

Most clusters give you (at least) two separate storage areas with very different rules:

Area	Use it for
<code>\$HOME</code>	The code itself (this repo). Usually small quota, backed up, <i>not</i> meant for run output.
<code>\$SCRATCH</code> / <code>\$WORK</code> (name varies)	Everything a simulation writes. Usually large quota, <i>not</i> backed up, and some clusters auto-delete old files here — treat it as temporary.

Run `ls -la $HOME` and check your cluster’s documentation (or ask IT) for the scratch area’s actual environment variable if `$SCRATCH` isn’t it.

Recommended layout — one directory per run, never reused, so nothing from an old run silently mixes with a new one:

```
$SCRATCH/axion_runs/
  2026-09-21_N512_seed1/
    params.txt          # the exact parameter file used
    slurm.sh            # the exact submission script used
    run.log             # this run's console output
    network_scalars.dat # small diagnostics (keep)
    axion_spectrum.dat  # small diagnostics (keep)
    axion_projection.dat # small diagnostics (keep, if enabled)
    plt00000/ plt00100/ ... # plotfiles, if amr.plot_int is set (usually large
-- see Sec. 9)
    chk00000/          # checkpoints, if amr.check_int is set
```

Copying `params.txt` and `slurm.sh` into the run’s own directory (rather than only keeping one shared copy) means that six months from now you can look at any run directory and know exactly what produced it, with no guessing from memory or from a shared file that’s since been edited.

5 Step 4: Set up a run — the parameter file

Every run is controlled by one plain-text file (e.g. `params.txt`), lines of the form `key = value`. Existing examples live in `AxionStrings/params_*.txt` — copying the closest existing one and editing it is usually easier than writing one from scratch.

5.1 The parameters you’ll actually touch

`axion_strings.c0`

Which physical system: 0 = physical strings (the real thing), 1 = “fat” string (cheaper, approx-

imate), 2 = “Moore” mode. Start with 0 unless told otherwise.

`axion_strings.N`

Overall box size / resolution target. This is the single biggest lever on both physical realism and cost — see Section 5.5.

`axion_strings.N1, N2`

Leave these at 1.0 unless someone has specifically told you otherwise — they fix two resolution targets (“horizon fits in the box” and “string core is resolved”) that the project has already settled on.

`axion_strings.log_mr_over_h_i`

How far into the simulation’s natural time range the run *starts*. Bigger box (N) sizes can afford to start later (a bigger number) without the string network being unrealistically fresh. 2.0 is a safe, already-used starting point.

`amr.max_level`

How many levels of automatic refinement to allow (0 = no refinement, a plain uniform grid). Higher is more realistic *and* usually much cheaper for the same finest resolution (adaptive refinement only pays for fine cells where they’re needed) — see the timing note in Section 5.5. 2 is a good, validated starting point.

`axion_strings.ic_mode`

How the simulation starts. `fourier_relaxed` (a randomised starting field, settled into a realistic tangle before the “real” run begins) is what production runs use — it needs the four companion parameters just below, all of which are *required* (the run refuses to start without them, there is no built-in fallback).

`axion_strings.pre_evolution.seed`

The random seed for that starting field. **This is your ensemble knob** — run the same parameter file with different seeds to get independent realisations of the same physics (see the SLURM array example in Section 7).

`amr.plot_int`

How often (in steps) to write a full 3D plotfile. These are large. Leave at -1 (off) unless you specifically want to visualise the field; the physics diagnostics you actually analyse (`network_scalars.dat` etc.) are controlled separately below and are cheap.

`axion_strings.output_first_log_mr_over_h`

Where in the run to start writing rows to the small diagnostics files. There’s no built-in default — in practice this is almost always set equal to `log_mr_over_h_i` just above (start recording from the very beginning of the main run).

`axion_strings.output_delta_log_mr_over_h`

How often, after that, to write another row. 0.1--0.2 is a typical spacing.

5.2 The `fourier_relaxed` companion parameters

These four are required whenever `axion_strings.ic_mode = fourier_relaxed` (the recommended, default choice above). All four have settled, essentially fixed typical values, or — for `gamma` — are simply left unset so the program derives it for you.

`axion_strings.pre_evolution.gamma`

Leave this unset. The program derives it automatically as $1/dx_{\text{base}}$ (the coarsest-level grid spacing) and prints what it derived as part of its startup box plan — this is the resolution the

relaxation phase needs to run at so the string cores it produces are resolved consistently with what the main run's own grid looks like the moment it starts (relies on `log_mr_over_h_i` being below the level-1 threshold printed just above it, i.e. the main run genuinely starting on the coarsest grid alone — the program checks this itself and refuses to start if it isn't true, rather than silently relaxing at the wrong resolution). You can still set it by hand if you have a specific reason to (e.g. experimenting with a different relaxation resolution) — the program then checks your value against what it would have derived and prints a warning (not an error — the run still proceeds with your value) if they disagree, so an accidental stale value from a previous box size doesn't go unnoticed.

`axion_strings.pre_evolution.xi_target`

Stop relaxing once the (measured) string density first drops to this value. **Typical value:** 1.0 (used throughout the project's own validated examples; some production runs instead target a lower value like 0.7 for a more settled starting tangle at the cost of a longer relaxation phase).

`axion_strings.pre_evolution.k_max_over_mr`

The highest wavenumber (in units of m_r) populated in the random starting field. **Typical value:** 64, used in every validated example at `amr.max_level=2`. If you're testing a *much* coarser base grid or a deeper hierarchy than those examples, this is worth being cautious with — `docs/STATUS.md` documents a specific pathology where too high a value here (relative to how coarse the base grid is) over-tags the whole domain for refinement; if a test run's very first regrid tags close to 100% of the domain unexpectedly, this is the first parameter to suspect.

`axion_strings.pre_evolution.mean_square_variance`

How much random power to start with. **Always** 0.5 in this project's convention (chosen so the field's total mean-square amplitude works out to the natural unit value) — not something to change casually.

5.3 The handoff from relaxation to the main run

Relaxation and the main run use *different* background schedules — relaxation deliberately holds m_r/H fixed so the network can settle onto an attractor quickly, while the main run lets m_r/H grow (that growth is the whole point of the simulation). This means the two schedules' $\lambda(\tau)$ (and hence the string core's natural comoving width) generically disagree at whatever moment relaxation happens to finish, and switching between them with no further care excites a measurable, spurious transient: an oscillatory exchange between the radial mode's kinetic and potential energy, and a bump in the axion spectrum near the old core scale that only slowly migrates towards the new one. Neither is removed by the usual masking scheme.

`axion_strings.pre_evolution.handoff_transition_n_periods` (**default** 3.0, on by default) fixes this by blending λ and `curvature_term_coeff` smoothly from their relaxation-phase values to the main schedule's own values over this many main-schedule core-oscillation periods, instead of switching instantaneously. Direct comparison (relaxing the exact same random field, with and without this smoothing) found the transient's amplitude drops by several times with the default of 3; doubling it to 6 bought no further improvement, so 3 was kept as the standing default rather than left as something you need to opt into. You do not need to set this yourself — it is listed here so you know it exists and what it's protecting against. Set it to 0 only if you have a specific reason to reproduce the old instantaneous-jump behaviour.

5.4 A note on `amr.max_grid_size`

This one doesn't change the physics at all, only how the domain is chopped up for parallel processing — but it needs to roughly match how many MPI ranks you plan to run with (Section 7). A smaller `amr.max_grid_size` makes more, smaller chunks, letting you usefully spread the coarse level across more ranks. As a rule of thumb: (grid size at that level) $\div$ `max_grid_size`, cubed, should be at least as big as your *rank* count — specifically ranks, not total CPUs: each rank's own threads (Section 3.4) already split the cells *within* whatever chunks that rank owns, so this box count doesn't need to grow with `--cpus-per-task` the way it would need to grow with `--ntasks`.

5.5 Scaling up to a bigger test

1. Take the closest existing `params_*.txt` as a starting template (e.g. `params_full_test_640.txt`).
2. Raise `axion_strings.N`. Leave `N1/N2` alone.
3. If your template has an `axion_strings.pre_evolution.gamma` line (every existing example does), delete it — it was computed for the *old* box size and won't match the new one, and while the program will still run and only warn about the mismatch (Section 5.2), the relaxation phase would silently run at the wrong resolution if you missed that warning. Leaving `gamma` out entirely sidesteps the issue and lets the program derive the right value for you.
4. **Always do a very short test run first** (see the box below) before committing a multi-hour job to the queue.

Watch out

The program prints its own “box plan” (grid spacing, box size, refinement schedule) in the first few lines of output, every time it starts, *before* it starts stepping. Always read those first few lines after raising `N` and check nothing looks surprising (in particular, an unexpectedly coarse grid spacing has previously caused a run to tag far more of the domain for refinement than intended, quietly making a run much more expensive than planned). A 2–3 minute local test with a handful of steps (`evolution.max_steps=50` added on the command line) is enough to see this printout and catch a misconfiguration before it costs a multi-hour cluster allocation.

5.6 A worked example: a moderately big cluster run

`AxionStrings/params_cluster_512base_2level.txt` is a complete, ready-to-run example at a scale genuinely worth a cluster (not just a laptop test): a 512^3 base grid with 2 refinement levels ($N_{\text{effective}} = 2048$), physical strings, every optional diagnostic in Section 6's “Extra diagnostics” turned on so you can see all of them wired up in one place, and the regrid-buffer mechanism from Section 6.1 used instead of hand-set values. Its own header comment shows the full box-plan arithmetic worked by hand, the same way this section asks you to check it — and it has actually been smoke-tested (a few steps, confirmed no crash and an exact match between the hand-worked numbers and what the program itself printed) before being included here, not just written and assumed correct.

Use it directly, or — more usefully — as the template Step 1 above refers to: copy it, change `axion_strings.N` for your own target size, delete the (still-present, now stale) comment block doing the old box-plan arithmetic by hand, and let the short test run's own printout give you the new numbers instead.

6 Less commonly changed options

Everything in this section has a sensible default and most runs never need to touch it — skip it on a first read and come back if you have a specific reason to (a run behaving oddly, or someone asking you to try a specific value).

6.1 How often refinement is re-checked, and its safety buffer

The adaptive mesh doesn't re-decide where to refine on every single step (that would be wasteful) — it re-checks periodically, then grows a safety buffer of extra-refined cells around every flagged region so that nothing genuinely fast-moving can slip out of the fine grid in between checks.

There are exactly two parameters to set here, together, and nothing else in this subsection needs touching once you have:

`axion_strings.tagging.regrid_interval_steps`

How often, in steps, the simulation re-decides where to refine. **Typical value:** 2 (matching every one of the project's own validated example runs). Smaller tracks the strings more tightly (fewer stale/wasted fine cells) but costs more time spent re-deciding; larger is cheaper bookkeeping but requires a wider safety buffer (derived automatically below), meaning more volume held at finer resolution than strictly necessary.

`axion_strings.tagging.buffer_safety_factor`

A multiplier ≥ 1.0 on top of the bare-minimum buffer width the code derives from `regrid_interval_steps` above. **Default:** 1.0 (the bare minimum) — raise it (e.g. to 1.5 or 2.0) only for extra headroom when trying a new, not-yet-trusted combination of parameters, at the cost of refining a bit more volume than the bare minimum.

Watch out

Setting these two is an *alternative* to setting `amr.regrid_int/amr.n_error_buf` yourself, not an addition to it — once `regrid_interval_steps` is set, the run derives both of those for you automatically and prints what it derived (and why) in its startup output, right alongside the box plan (worth reading alongside the box-plan check in Section 5.5's warning box). **Don't also set `amr.regrid_int/amr.n_error_buf` by hand in the same parameter file** — if you do, the run cross-checks them against what it derived and stops with an error rather than silently picking one. The rest of this box explains what those two lower-level parameters mean, only for the (uncommon) case where you want to bypass the mechanism above entirely and set them directly instead:

`amr.regrid_int`

Steps between refinement re-checks, set directly. -1 on a single level disables regriding entirely (used by the wave-packet boundary tests in `AxionStrings/analysis/wave_boundary_test/` to hold a grid fixed).

`amr.n_error_buf`

The safety buffer width itself, in cells, set directly.

6.2 How sensitive the refinement trigger is (off by default)

What actually finds strings for refinement, for context on the two optional parameters below: the tagger's *primary* criterion is an exact, parameter-free topological test — does the field's phase wind by a full $\pm 2\pi$ loop around a cell's corner plaquette? That's an integer (a string

pierces this cell or it doesn't), not a threshold on a continuous quantity, so it has no “units” question and nothing for you to set — it's always on. *Which levels are even allowed to exist yet* is controlled separately, by the schedule (`log_mr_over_h_i` and the box-plan-derived level-addition thresholds); the two parameters below don't touch that either. Both default to effectively infinite (i.e. off) and were found, in earlier testing, to flag essentially the entire domain if turned on carelessly — leave them alone unless you have a specific reason to experiment with gradient-based tagging, and if you do, read `StringTagger.hpp`'s own header comment first for the full reasoning.

`axion_strings.tagging.gradient_threshold`

A *second*, independent, optional trigger — despite the name, actually a *Laplacian* (curvature) test, applied separately to each of the field's two real components (ψ_1, ψ_2 ; not a decomposed “axion” or “phase” quantity specifically — that split is what the next parameter does). A cell is tagged if $dx^2 |\nabla^2 \psi_i|$ exceeds this threshold, for either component. **Units: dimensionless** — dx^2 (comoving length²) exactly cancels the Laplacian's $1/\text{length}^2$, leaving a pure number of order “how much ψ curves across one cell, relative to its own natural $O(1)$ scale” (ψ is normalised by the vacuum value $v = 1$ in this code's units). Buschmann et al. 2108.05368's own value is 0.04, but this project's own testing found that value tags essentially the whole domain for its field statistics — not a starting point to trust as-is.

`axion_strings.tagging.radial_gradient_threshold`

A *third*, independent, optional trigger, this one genuinely radial: $r = |\psi|$, the field's amplitude alone (properly decomposed away from the phase direction, not just combining $\nabla\psi_1/\nabla\psi_2$ naively). A cell is tagged if $dx |\nabla r|$ exceeds this threshold. **Units: also dimensionless** (same cancellation, one power of dx against ∇r 's $1/\text{length}$) — but this is a *first* derivative of a different, derived quantity, not the same operation as `gradient_threshold`'s Laplacian of the raw components. Both end up dimensionless on the same rough scale (“per-cell change in units of the field's own $O(1)$ scale”), but a given numeric value doesn't mean the same thing for both — don't assume tuning one tells you anything about the other.

6.3 Domain decomposition, the other half

`amr.blocking_factor`

Every box's size (in each dimension) must be a multiple of this. Works alongside `amr.max_grid_size` (Section 5.5) to control how the domain is chopped up for parallel ranks — the project's own example files use 16 or 32; there's rarely a reason to deviate unless you're chasing a specific load-balancing problem.

6.4 Timestep safety margin

`evolution.dt_multiplier`

The CFL safety factor: how large a timestep to take as a fraction of the finest active grid spacing. 0.2 is the project's standard, conservative choice. Raising it makes runs faster but risks numerical instability — not something to change without a specific reason and a stability check.

6.5 Cosmological expansion rate

`axion_strings.a_inv`

Which cosmological era the box expands through (2.0 = radiation domination, the project's standard choice throughout). Only change this if you specifically intend to simulate a different era — it changes the box-planning formulas, not just a cosmetic label.

6.6 Extra diagnostics

These add real cost (the spectrum needs an FFT of the whole level-0 grid; projections write an extra file per snapshot), which is why they default off.

`axion_strings.masking.scheme`, `axion_strings.masking.threshold`

`scheme` defaults to A (unscreened) if left unset. Set it to B (with a `threshold` in $(0, 1)$ — **required** in that case, no default) to additionally get *screened* energy/tension numbers — “screened” excludes a region around each string core from the energy sum, which some analyses want and others don’t. Values 0.8, 0.9 and 0.95 have all been used by this project at different times without settling on one, so treat any particular value as a choice to state explicitly in your own notes, not an established default. `axion_strings.compute_spectrum` (below) requires `scheme B`.

`axion_strings.compute_spectrum`

Set to 1 to write `axion_spectrum.dat` (needed for the spectrum panel of Section 10’s standard plots). Requires `axion_strings.masking.scheme = B` (just above).

`axion_strings.save_projection`

Set to 1 to additionally save a 2D projection of the field at each diagnostic snapshot, useful for visualising the string network’s evolution over time rather than only its summary statistics.

6.7 Checkpointing and restart

`amr.check_int`

How often (in steps) to write a full checkpoint you can restart from. -1 (the default in every example file) turns it off. Turn it on for any run that might not finish inside one job’s walltime — see the warning box in Section 8. Restarting from a checkpoint changes how you invoke the executable; ask if you need this and it isn’t already covered by an example you’ve found.

7 Step 5: Write and submit the SLURM script

Below is a template adapted from an existing working example for DESY’s Maxwell cluster. If you’re on a different cluster, the `--partition`, module names, and `OMPI_MCA_btl` line are the parts most likely to need changing — ask your cluster’s documentation or support for the local equivalents.

```
#!/bin/bash -l
#SBATCH --job-name=axion_test
#SBATCH --output=%x_%A_%a.log
#SBATCH --array=1-4                # one job per seed: 4 independent realisations
#SBATCH -N 1                       # nodes
#SBATCH --ntasks=8                 # MPI ranks (see amr.max_grid_size note, Sec. 5.3)
#SBATCH --cpus-per-task=4          # OpenMP threads per rank, Sec. 3.4 -- 8x4 = 32
    CPUs total
#SBATCH --time=04:00:00            # walltime: raise for bigger/longer runs
#SBATCH --partition=maxcpu

module load mpi/openmpi-x86_64
export OMPI_MCA_btl=~vader,tcp,openib
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK # must match --cpus-per-task, Sec. 3.4

CODE_DIR=$HOME/axion_strings_adaptive/AxionStrings
RUN_DIR=$SCRATCH/axion_runs/$(date +%F)_test_seed${SLURM_ARRAY_TASK_ID}
mkdir -p "$RUN_DIR"
```

```
cp "$CODE_DIR/AxionStrings3d.gnu.FLOAT.MPI.OMP.ex" "$CODE_DIR/params.txt" "$RUN_DIR/"
cd "$RUN_DIR"
cp "$0" slurm.sh # keep a copy of the exact script that produced this run

mpirun -np 8 ./AxionStrings3d.gnu.FLOAT.MPI.OMP.ex params.txt \
    axion_strings.pre_evolution.seed=${SLURM_ARRAY_TASK_ID} \
    --bind-to none \
    > run.log 2>&1
```

A few things worth calling out explicitly:

- `#SBATCH --array=1-4` is what runs 4 independent realisations (different random seeds) as one job submission — each array element gets its own `$SLURM_ARRAY_TASK_ID`, which the script both uses as the seed *and* as part of the run directory name, so the four never collide.
- Parameters after the parameter-file name on the `mpirun` line (`axion_strings.pre_evolution.seed=...`) *override* whatever's in `params.txt` — handy for exactly this kind of per-job tweak without needing four separate parameter files.
- `--bind-to none` matters more here than it would without threads: it stops Open MPI pinning each rank to a single CPU, which would otherwise trap that rank's 4 threads fighting over one core instead of spreading across the 4 `--cpus-per-task` reserved for it.
- `--ntasks` is the number of MPI ranks — it should match `-np` on the `mpirun` line. `--cpus-per-task` is threads *per rank*, and only does anything useful together with a `USE_OMP=TRUE` build and a matching `OMP_NUM_THREADS` (Section 3.4, both already in this template) — if you ever build *without* `USE_OMP=TRUE`, set `--cpus-per-task` back to 1, or the extra CPUs just sit idle.
- Everything writes into a fresh, timestamped run directory under `$SCRATCH`, matching Section 4.

Submit it, and check it's queued:

```
sbatch slurm.sh
squeue -u $USER
```

8 Step 6: Watch it run

Once a job starts, its log file (`run.log`, from the redirection in the script above) fills in real time:

```
tail -f $SCRATCH/axion_runs/<run directory>/run.log
```

A healthy run shows repeating blocks like:

```
[Level 0 step 214] ADVANCE at time 3.02 with dt = 0.0142
[Level 0 step 214] average evolution speed = 812.4 code units/h
[Level 0 step 214] Advanced 2097152 cells
```

average evolution speed is the code's own built-in throughput readout — if it's dropping sharply and staying low, the simulation has likely started refining a large fraction of the domain (check the `REGRID` lines just above for how much). `network_scalars.dat` (in the same run directory) fills in alongside it with the physics summaries and can be watched the same way (`tail -f`) for a terser, physics-only view.

Watch out

If a job hits its `--time` limit before finishing, SLURM kills it — it does not pause and resume automatically. If you expect a run might need longer than one job's walltime, turn on checkpointing (`amr.check_int`, an interval in steps) so you can resubmit and continue from the last checkpoint rather than losing the run. This is not covered further here — ask if you need it, since it changes the submission script slightly (the executable needs to be told which checkpoint to restart from).

9 Step 7: Bring the results home

You almost always want only the small diagnostic files, not the plotfiles (each plotfile is the entire 3D field on disk and can be many gigabytes; the diagnostics already capture everything the standard analysis plots need). From your own machine:

```
rsync -avz \
  you@cluster.example.edu:$SCRATCH/axion_runs/<run directory>/network_scalars.dat \
  you@cluster.example.edu:$SCRATCH/axion_runs/<run directory>/axion_spectrum.dat \
  you@cluster.example.edu:$SCRATCH/axion_runs/<run directory>/axion_projection.dat \
  ~/axion_runs/<run directory>/
```

(Drop any of the three filenames that weren't produced by your particular run — e.g. `axion_spectrum.dat` only appears if `axion_strings.compute_spectrum=1` was set.) It's worth also grabbing `run.log`, `params.txt` and `slurm.sh` from the same directory — tiny, and they're the record of exactly what produced the data.

10 Step 8: Analyse it locally

The repository's `AxionStrings/analysis/` scripts take those diagnostic files directly and produce the project's standard plots. On your own machine, with the repository's Python environment active (see the analysis folder's own `README.md` for setup if you haven't used it before):

```
cd axion_strings_adaptive/AxionStrings/analysis
python3 plot_run_summary.py \
  ~/axion_runs/<run directory>/network_scalars.dat \
  ~/axion_runs/<run directory>/axion_spectrum.dat \
  ~/axion_runs/<run directory>/plots_out \
  <a_inv> <c0> <N> [optional title tag]
```

This produces the standard set: string density ξ , energies (raw and normalised), string tension, and the axion spectrum, both raw and normalised. Fill in `<a_inv>`, `<c0>`, `<N>` with the exact values from the `params.txt` you ran with (they affect the unit conversions, so they need to match, not just be "close").

If you don't have an `axion_spectrum.dat` (i.e. you didn't turn spectrum output on), pass `None` in its place — the script skips the spectrum panel and still produces the rest.

11 Quick reference

Task	Command
Get code onto cluster	<code>git clone --recurse-submodules ...</code> Or <code>rsync -avz ...</code>
Build	<code>make COMP=gnu FFTW_HOME=...USE_OMP=TRUE -j8</code>
Submit a run	<code>sbatch slurm.sh</code>
Check queue	<code>squeue -u \$USER</code>
Watch a running job	<code>tail -f run.log</code>
Cancel a job	<code>scancel <jobid></code>
Bring results home	<code>rsync -avz user@cluster:.../*.dat /axion_runs/.../</code>
Make the standard plots	<code>python3 plot_run_summary.py ...</code>

If something in this guide doesn't match what you see (a module name, a file that isn't there, an error message), that's much more likely to be a cluster-specific detail this guide couldn't anticipate than a mistake on your end — check the cluster's own documentation for the specific piece (module names and partition names are the most commonly different), and the rest of the workflow should carry over unchanged.